

:ON UBUNTU :

Step 1: Install Nginx

First, make sure your system is up to date:

```
sudo apt update && sudo apt upgrade -y
```

Then install Nginx:

```
sudo apt install nginx -y
```

Start and enable Nginx to run on boot:

```
sudo systemctl start nginx  
sudo systemctl enable nginx
```

Check if Nginx is running:

```
systemctl status nginx
```

Now, on your Windows machine, open Chrome and visit your Vagrant VM's IP address (e.g., <http://192.168.56.101>).

You should see the Nginx default welcome page.

Step 2: Deploy a Single Website

1 Create a Directory for Your Website

Let's create a directory for example.com:

```
sudo mkdir -p /var/www/example.com/html
```

Give permissions:

```
sudo chown -R $USER:$USER /var/www/example.com/html  
sudo chmod -R 755 /var/www/example.com
```

2 Create an Index Page

```
echo "<h1>Welcome to Example.com</h1>" | sudo tee  
/var/www/example.com/html/index.html
```

3 Create a Nginx Virtual Host Configuration

Create a new configuration file for your website:

```
sudo nano /etc/nginx/sites-available/example.com
```

Add the following content:

```
server {  
    listen 80;  
    server_name example.com www.example.com;  
    root /var/www/example.com/html;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

④ Enable the Configuration

Create a symbolic link to enable the site:

```
sudo ln -s /etc/nginx/sites-available/example.com  
/etc/nginx/sites-enabled/
```

Test Nginx:

```
sudo nginx -t
```

Reload Nginx:

```
sudo systemctl reload nginx
```

⑤ Access the Website

If you don't have a domain, add an entry to your **Windows hosts file** (C:\Windows\System32\drivers\etc\hosts):

```
192.168.56.101 example.com
```

Now, visit <http://example.com> in your Windows Chrome browser.

Step 3: Deploy Multiple Websites

① Create Directories for Each Website

```
sudo mkdir -p /var/www/site1.com/html
```

```
sudo mkdir -p /var/www/site2.com/html
```

Set permissions:

```
sudo chown -R $USER:$USER /var/www/site1.com/html  
/var/www/site2.com/html
```

```
sudo chmod -R 755 /var/www/
```

② Create Index Pages

```
echo "<h1>Welcome to Site1</h1>" | sudo tee  
/var/www/site1.com/html/index.html
```

```
echo "<h1>Welcome to Site2</h1>" | sudo tee  
/var/www/site2.com/html/index.html
```

③ Create Virtual Host Configurations

For Site1: `sudo nano /etc/nginx/sites-available/site1.com`

```
server {  
  
    listen 80;  
  
    server_name site1.com www.site1.com;  
  
    root /var/www/site1.com/html;  
  
    index index.html;  
  
    location / {  
  
        try_files $uri $uri/ =404;  
  
    }  
  
}
```

For Site2: `sudo nano /etc/nginx/sites-available/site2.com`

```
server {  
  
    listen 80;  
  
    server_name site2.com www.site2.com;  
  
    root /var/www/site2.com/html;  
  
    index index.html;  
  
    location / {  
  
        try_files $uri $uri/ =404;  
  
    }  
  
}
```

4 Enable the Sites

```
sudo ln -s /etc/nginx/sites-available/site1.com  
/etc/nginx/sites-enabled/
```

```
sudo ln -s /etc/nginx/sites-available/site2.com  
/etc/nginx/sites-enabled/
```

Test Nginx: `sudo nginx -t`

Reload Nginx: `sudo systemctl reload nginx`

5 Configure Windows to Access Multiple Websites

Edit your Windows hosts file

(C:\Windows\System32\drivers\etc\hosts):

```
192.168.56.101 site1.com
```

```
192.168.56.101 site2.com
```

Now, visit <http://site1.com> and <http://site2.com> in Chrome.

Hosting Multiple Websites on EC2 Using Nginx with Paths (Without Domains)

You want to host multiple websites on an EC2 instance and access them via your public IP like this:

- `http://<EC2-IP>/site1` → Shows Site 1
- `http://<EC2-IP>/site2` → Shows Site 2
- You can add N number of sites without setting up domains.

1. Create Directories for Each Site

Let's create separate directories for `site1` and `site2` inside Nginx's default web root (`/var/www/html/`):

```
sudo mkdir -p /var/www/html/site1
```

```
sudo mkdir -p /var/www/html/site2
```

Give proper ownership and permissions:

```
sudo chown -R $USER:$USER /var/www/html/
```

```
sudo chmod -R 755 /var/www/html/
```

2. Create Index Pages for Each Site

Create an `index.html` file for each site:

```
echo "<h1>Site 1</h1>" | sudo tee /var/www/html/site1/index.html
```

```
echo "<h1>Site 2</h1>" | sudo tee /var/www/html/site2/index.html
```

3. Configure Nginx to Serve Multiple Sites via Paths

Edit the **default Nginx configuration**:

```
sudo nano /etc/nginx/sites-available/default
```

Modify it to include the following:

```
server {  
  
    listen 80;  
  
    server_name _;  
  
    root /var/www/html;  
  
    index index.html;  
  
    location / {  
  
        try_files $uri $uri/ =404;  
  
    }  
  
    location /site1 {  
  
        index index.html;  
  
        alias /var/www/html/site1/;  
  
    }  
  
    location /site2 {  
  
        index index.html;  
  
        alias /var/www/html/site2/;  
  
    }  
}
```

4. Restart Nginx

Test the configuration: `sudo nginx -t`

If everything is fine, restart Nginx: `sudo systemctl reload nginx`

Access Your Sites from Any Device!

Now, open a browser on **any device** and visit:

- `http://<EC2-Public-IP>/site1` → Shows Site 1
- `http://<EC2-Public-IP>/site2` → Shows Site 2

✓ **No need for domain setup!** You can add more sites using the same metho

Deploying Nginx Websites on RHEL/CentOS (Single & Multiple Sites)

❶ Install & Start Nginx on RHEL/CentOS

Step 1: Install Nginx: `sudo yum install -y nginx`

Start and enable Nginx:

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

By default, Nginx serves content from `/usr/share/nginx/html/`

(not `/var/www/html/` like in Ubuntu).

❷ Deploy a Single Website

We will host a single website at `http://<EC2-IP>/`.

Step 2: Create Website Directory

```
sudo mkdir -p /usr/share/nginx/site1
```

Set permissions:

```
sudo chown -R $USER:$USER /usr/share/nginx/site1
```

```
sudo chmod -R 755 /usr/share/nginx/site1
```

Step 3: Create a Sample Web Page

```
echo "<h1>Welcome to Site 1</h1>" | sudo tee  
/usr/share/nginx/site1/index.html
```

Step 4: Configure Nginx to Serve the Site

Edit the default Nginx configuration:

```
sudo nano /etc/nginx/nginx.conf
```

Find the `server` block and modify it:

```
server {  
  
    listen 80;  
  
    server_name _;  
  
    root /usr/share/nginx/site1;  
  
    index index.html;  
  
    location / {  
  
        try_files $uri $uri/ =404;  
  
    }  
  
}
```

Save and exit.

Step 5: Restart Nginx

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

Now, visit <http://<EC2-IP>/>, and you should see "Welcome to Site 1".

③ Hosting Multiple Websites Using Paths (/site1, /site2)

Now, let's configure multiple sites using different URL paths like:

- `http://<EC2-IP>/site1`
- `http://<EC2-IP>/site2`

Step 1: Create Directories for Each Site

```
sudo mkdir -p /usr/share/nginx/site1
```

```
sudo mkdir -p /usr/share/nginx/site2
```

Set permissions:

```
sudo chown -R $USER:$USER /usr/share/nginx/
```

```
sudo chmod -R 755 /usr/share/nginx/
```

Step 2: Create Sample Pages

```
echo "<h1>Welcome to Site 1</h1>" | sudo tee  
/usr/share/nginx/site1/index.html
```

```
echo "<h1>Welcome to Site 2</h1>" | sudo tee  
/usr/share/nginx/site2/index.html
```

Step 3: Configure Nginx to Serve Sites by Paths

Edit **Nginx config**: `sudo nano /etc/nginx/nginx.conf`

Modify the `server` block:

```
server {  
  
    listen 80;  
  
    server_name _;  
  
    root /usr/share/nginx/html;  
  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
  
    location /site1 {  
        alias /usr/share/nginx/site1/;  
        index index.html;  
    }  
  
    location /site2 {  
        alias /usr/share/nginx/site2/;  
        index index.html;  
    }  
}
```

Save and exit.

Step 4: Restart Nginx

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

Step 5: Access Your Sites

Now, in your browser:

- <http://<EC2-IP>/site1> → Shows Site 1
- <http://<EC2-IP>/site2> → Shows Site 2

✓ You can add more sites using the same method!

Hosting Multiple Sites in Nginx on RHEL/CentOS Using [conf.d](#)

1 Create Directories for Each Website

Nginx on **RHEL/CentOS** serves files from `/usr/share/nginx/html/`, so we create directories for each site:

```
sudo mkdir -p /usr/share/nginx/site1
```

```
sudo mkdir -p /usr/share/nginx/site2
```

Set correct permissions:

```
sudo chown -R $USER:$USER /usr/share/nginx/
```

```
sudo chmod -R 755 /usr/share/nginx/
```

2 Create Index Pages for Each Site

```
echo "<h1>Welcome to Site 1</h1>" | sudo tee  
/usr/share/nginx/site1/index.html
```

```
echo "<h1>Welcome to Site 2</h1>" | sudo tee  
/usr/share/nginx/site2/index.html
```

3 Create Separate Nginx Config Files in **conf.d/**

Now, create **site1.conf** and **site2.conf** inside `/etc/nginx/conf.d/`.

```
sudo nano /etc/nginx/conf.d/site1.conf
```

Add the following:

```
server {  
  
    listen 80;  
  
    server_name _;  
  
    location /site1 {  
  
        alias /usr/share/nginx/site1/;  
  
        index index.html;  
  
    }  
  
}
```

Save and exit.

Now, create the config for Site 2:

```
sudo nano /etc/nginx/conf.d/site2.conf
```

Add:

```
server {  
  
    listen 80;  
  
    server_name _;  
  
    location /site2 {  
  
        alias /usr/share/nginx/site2/;  
  
        index index.html;  
    }  
}
```

Save and exit.

4 Restart Nginx

Check for errors: `sudo nginx -t`

If everything is fine, reload Nginx: `sudo systemctl reload nginx`

5 Access Your Sites

Now, open a browser and visit:

- `http://<EC2-IP>/site1` → Shows Site 1
- `http://<EC2-IP>/site2` → Shows Site 2